Fun with Filters and Frequencies!

OVERVIEW

Project 2

This project 2 explores different filters and frequencies.

My most important learning from this project is to always be diligent when writing the function to create the Gaussian and Laplacian stacks had a bit of a problem of not being able to reconstruct the images from the stack and this was because the lowest frequency was missing i Laplacian stack.

PART 1: FUN WITH FILTER

Part 1.1 - Convolutions from Scratch!

I have listed the results from the different convolutions below with the respective runtimes for each function (four for loops, two for loops, scipy.signal.convolve2d).

The code for the four loop implementation is as follows:

```
# four for loop implementation
def four_for_loop(img, filter):
  img = img.astype(np.float32)
  result = np.array([])
  for i in range(len(img)):
    line_result = np.array([])
    for j in range(len(img[0])):
      dot_product = 0
      for k in range(len(filter)):
        for l in range(len(filter[0])):
          if i+k < 0 or i+k >= len(img) or j+l < 0 or j+l >= len(img[0]):
            continue
          dot_product += img[i+k][j+l] * filter[k][l]
      line_result = np.append(line_result, dot_product)
    result = np.append(result, line_result)
  return result.reshape(len(img), len(img[0]))
```

For the two loops it is:

```
# two for loops
def two_for_loop(img, filter):
```

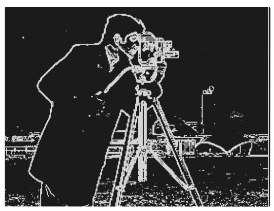
```
    img = img.astype(np.float32)
    result = np.array([])
    for i in range(len(img)-len(filter)):
      line_result = np.array([])
      for j in range(len(img[0])-len(filter[0])):
        dot_product = np.sum(img[i:i+len(filter), j:j+len(filter[0])] * filter)
        line_result = np.append(line_result, dot_product)
      result = np.append(result, line_result)
    return result.reshape(len(img)-len(filter), len(img[0])-len(filter[0]))
```

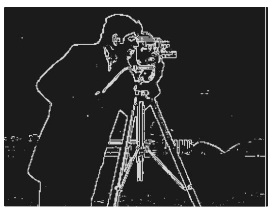
Filter	Result	Four for loops	Two for loops	scipy.signal.convolve2d
D_x		1.4462 seconds	2.4307 seconds	0.0044 seconds
D_y		1.5268 seconds	3.5698 seconds	0.0064 seconds
9x9 filter		15.0494 seconds	3.5590 seconds	0.0436 seconds

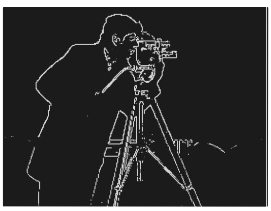
Part 1.2: Finite Difference Operator

The magnitude image and the edge images with different thresholds can be found below.


Magnitude image


Threshold: 0.1


Threshold: 0.2


Threshold: 0.3


Threshold: 0.4


Threshold: 0.5


Threshold: 0.6


Threshold: 0.7


Threshold: 0.8


Threshold: 0.9

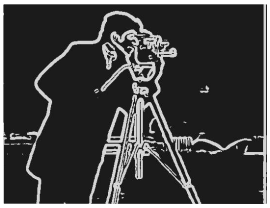
Based on the qualitative assessment, a threshold of about 0.3 seems the best.

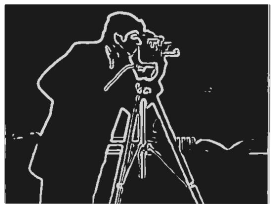
Part 1.3: Derivative of Gaussian (DoG) Filter

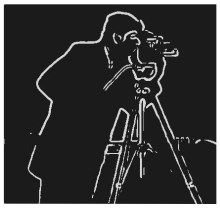
The gaussian blur image and the edge images with different thresholds can be found below.


Gaussian blur with kernel size 10 and sigma 2


Threshold: 0.1


Threshold: 0.2


Threshold: 0.3


Threshold: 0.4


Threshold: 0.5


Threshold: 0.6


Threshold: 0.7

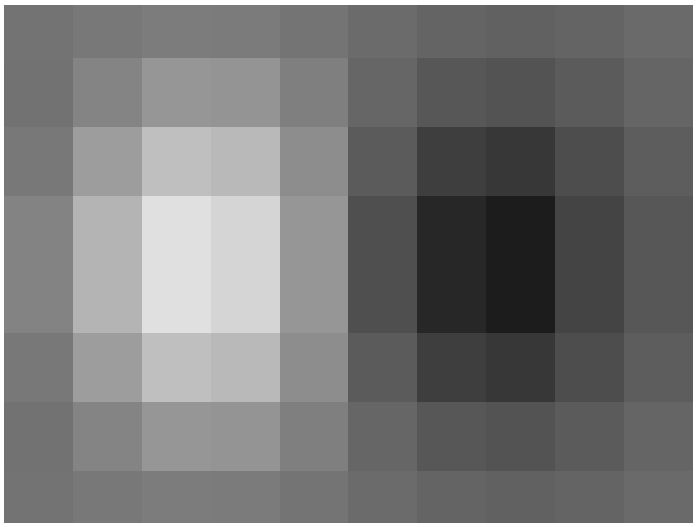

Threshold: 0.8

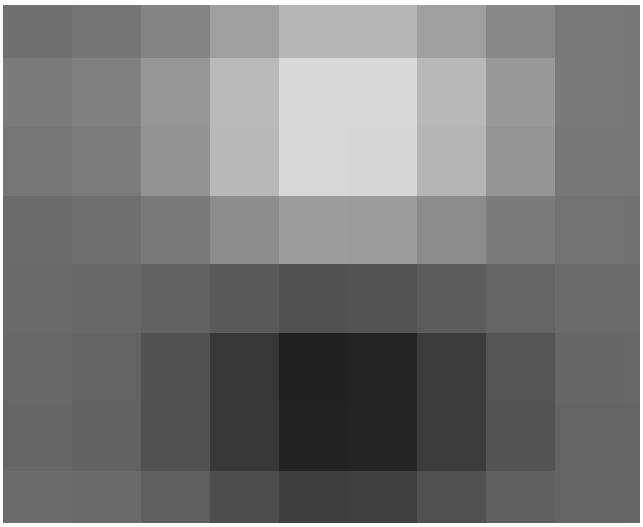

Threshold: 0.9

What differences do you see?

With a gaussian blur, the edges of the magnitude images are more pronounced and thicker. Consequently, the threshold images have a th and more distinct border as well. Now, a threshold of 0.2 gives good noise reduction while preserving most edges.

Now we can do the same thing with a single convolution instead of two by creating a derivative of gaussian filters. Convolve the gaussian v D_x and D_y and display the resulting DoG filters as images.


Gaussian * D_x


Gaussian * D_y

DoG filters as images

The gaussian can be convolved with D_x and D_y respectively to achieve the blurred edge detection in one step. The resulting images are t same, see below.


Blur and gaussian


Derivative of gaussian

Two-step (blur and gaussian) and one-step (derivative of gaussian) approach

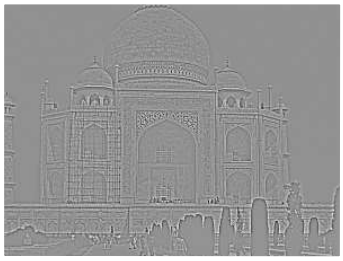
PART 2: FUN WITH FREQUENCIES!

Part 2.1: Image "Sharpening"

The image sharpening is performed on the three images below. In each example, you can see the unsharp and blurred image as well as the high frequency components and the resulting sharpened image.


Original unsharp image


Blurred image

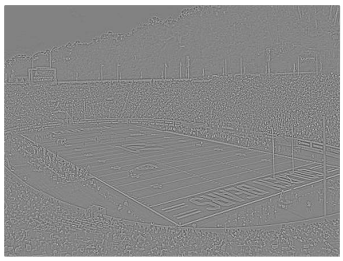

High frequency components


Sharpened image

Given Taj example


Original unsharp image


Blurred image


High frequency components


Sharpened image

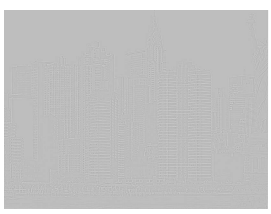
California memorial stadium

For the last example I took a sharp image, unsharpened it artifically to compare the sharpening algorithm to the original.


Original sharp image


Artificially unsharpened image


Blurred image


High frequency components


Sharpened image

Skyline of New York

What you can see at the New York example is that the re-sharpened image can recover some detail and contrast but does not perfectly ma the original sharp image. That is because the blur operation discards some high-frequency information that cannot be fully restored, sharpening can only amplify what remains.

Part 2.2: Hybrid Images

To determine the sigma values for the low and highpass, I manually inspected different values:

Sigma 2 = 30

Sigma 2 = 60

Sigma 2 = 90

Sigma 1 = 5

Sigma 2 = 30

Sigma 2 = 60

Sigma 2 = 90

Sigma 1 = 20

Sigma 2 = 30

Sigma 2 = 60

Sigma 2 = 90

Sigma 1 = 50

Another example

Car with high frequency paint

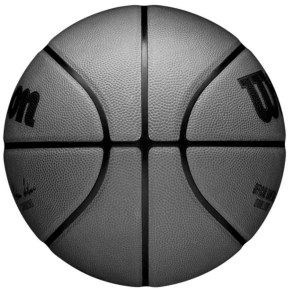
Plane with low frequency components

Hybrid result image

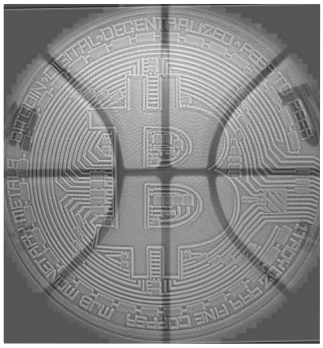
Sigma 1 = 1, Sigma 2 = 15

My favorite hybrid image is a mix of a basketball and a bitcoin.

Coin with high frequency paint



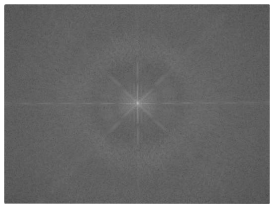
Ball with low frequency components



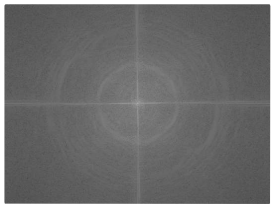
Hybrid result image

Sigma 1 = 1, Sigma 2 = 2

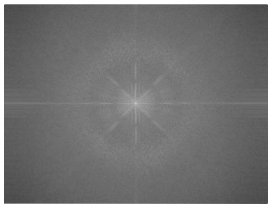
For the coinball the log magnitude of the Fourier transform of the two input images, the filtered images, and the hybrid image can be seen below.



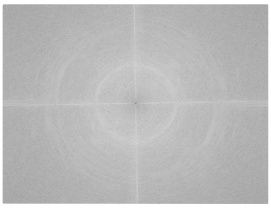
Baseketball log fft magnitude



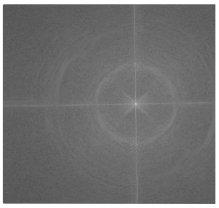
Coin log fft magnitude



Lowpassed baseketball log fft magnitude



Highpassed coin log fft magnitude

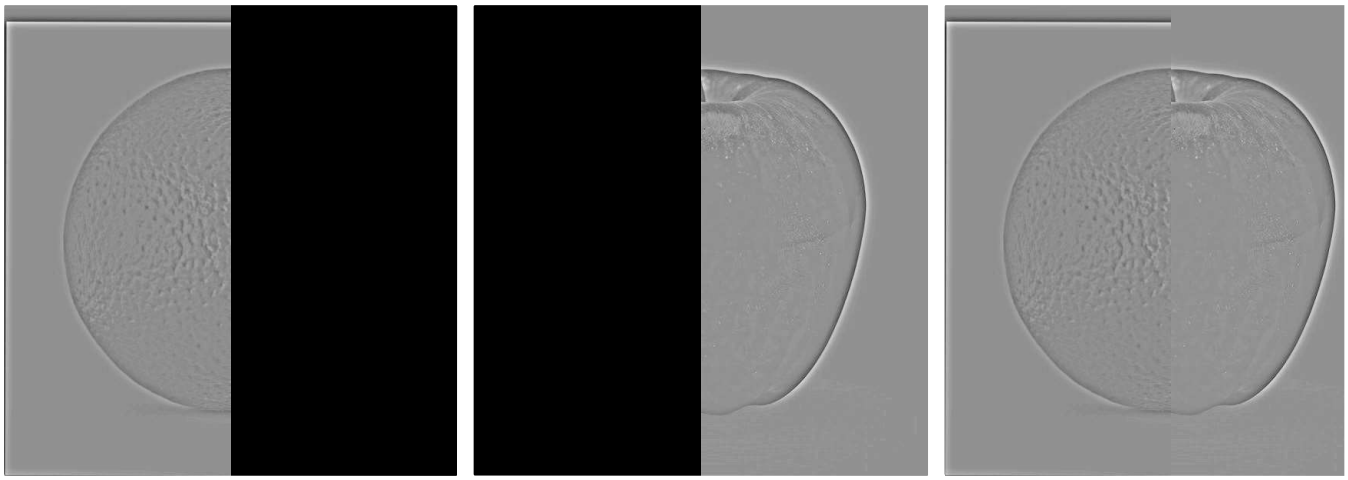


Highpassed coin log f magnitude

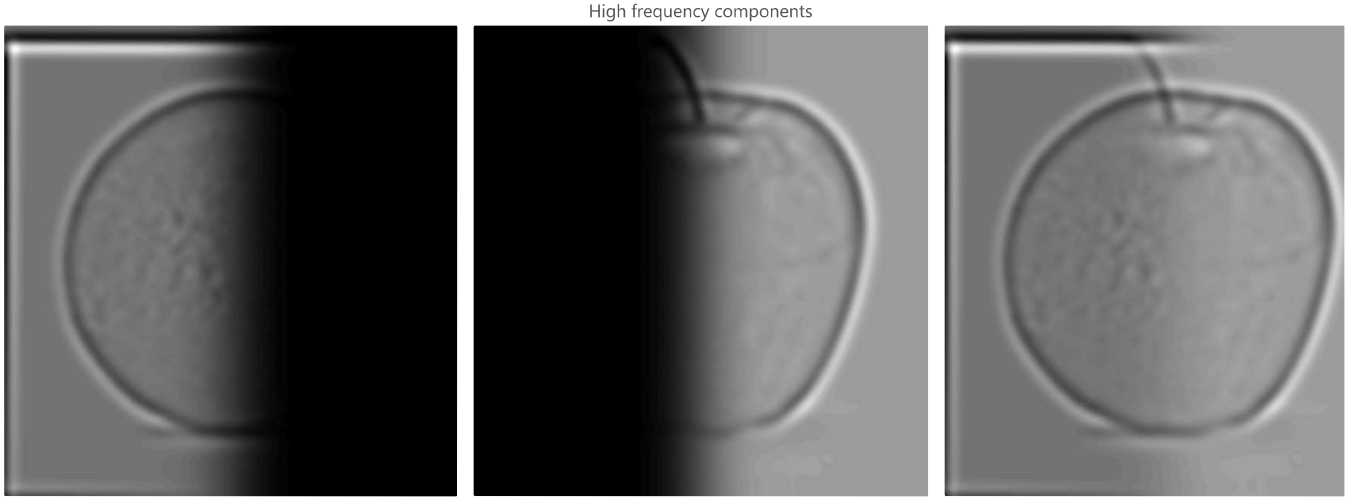
Log fft magnitude spectra for the different images

Part 2.3: Gaussian and Laplacian Stacks

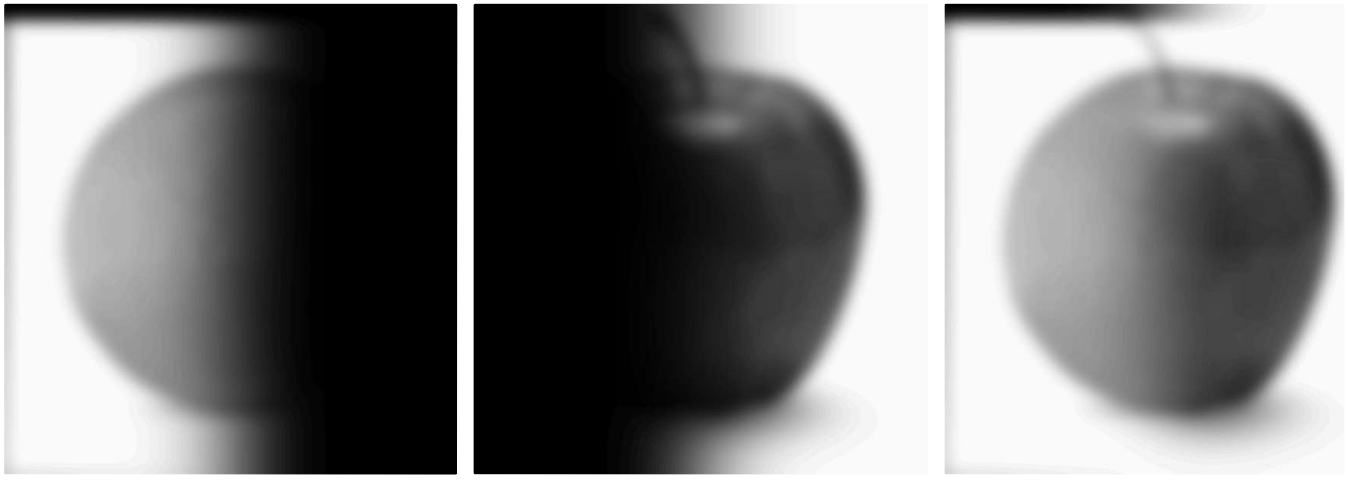
The different levels of the Laplacian stack are shown below for the apple, the orange and the blended image. I have used new apple and or images to also test the alignment.



High frequency components



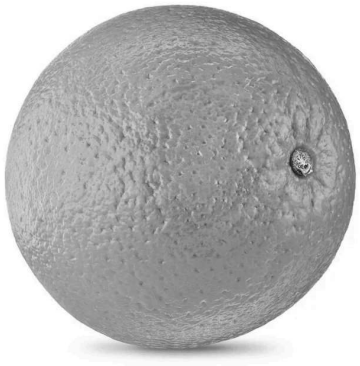
Mid frequency components



Low frequency components

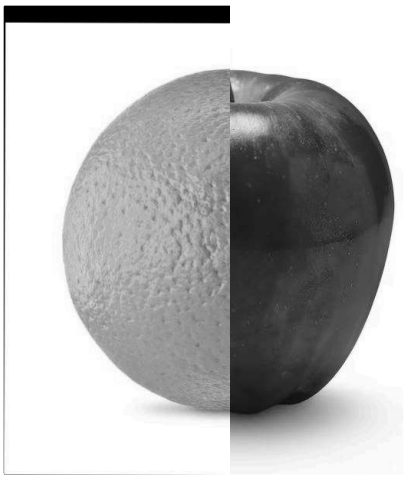
Images reconstructed from stack

The reconstructed blend from the apple and orange stack is substantially better than the side-by-side image below.



Orange

Apple



Blend

Raw images side-by-side

Part 2.4: Multiresolution Blending for another example

Another example is the blend of a hotdog with a banana.

Trivial combination

Blended combination

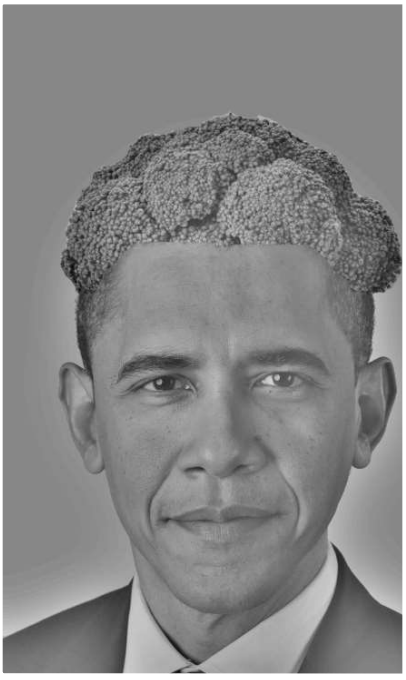
Combination of a hotdog and a banana

And my favorite example, where Obama is merged with a brocolli.

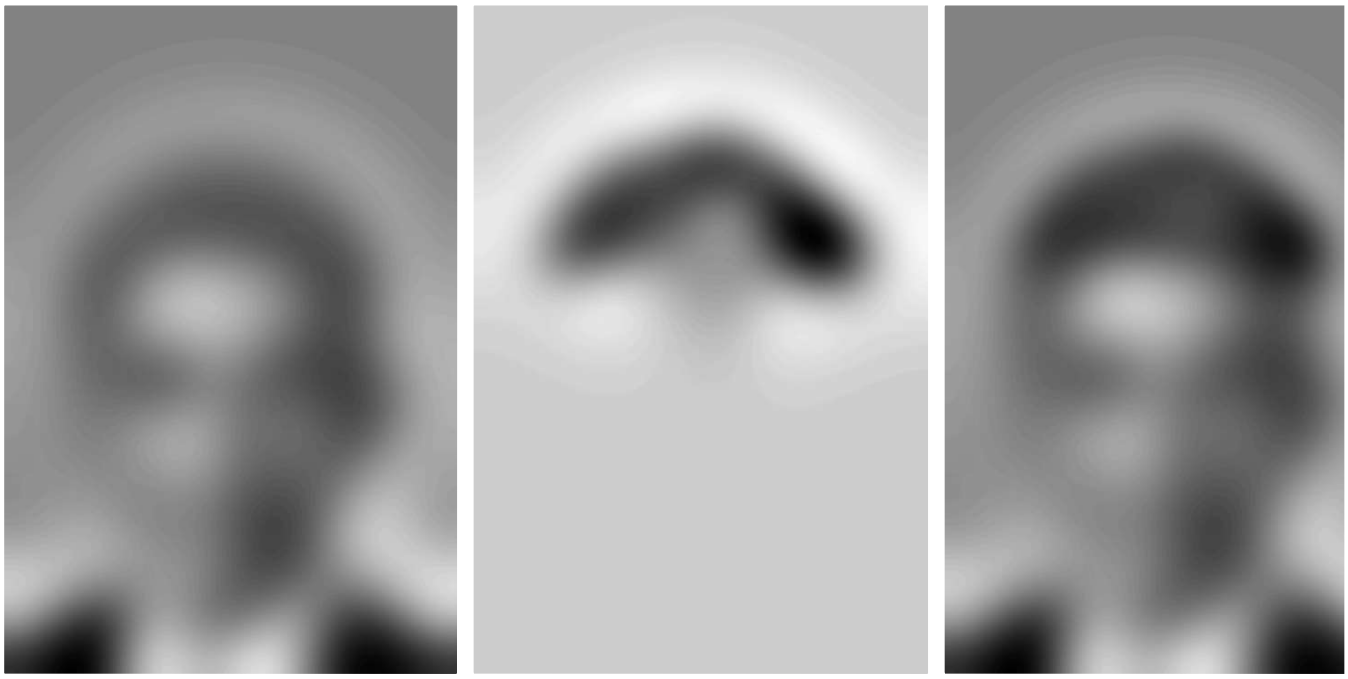
Trivial combination

Combination of Obama with a brocolli

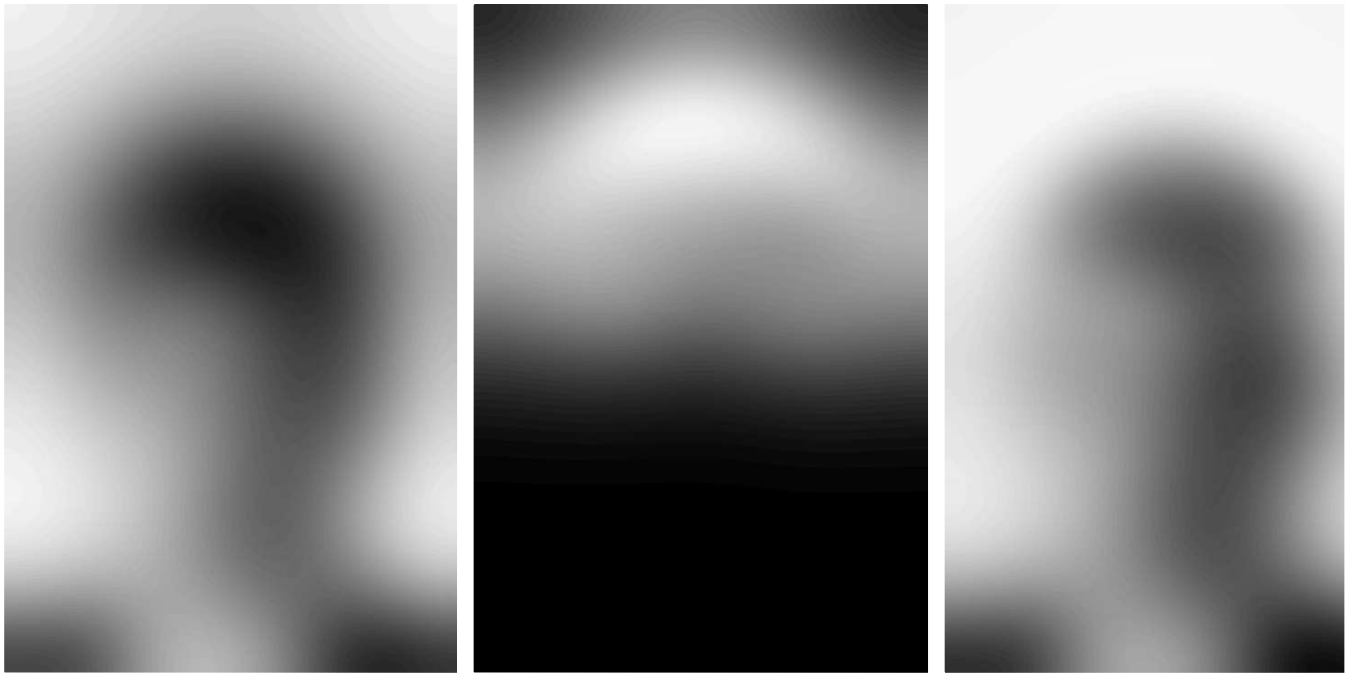
Blended combination



High frequency components



Mid frequency components



Low frequency components